

PetOS

SUMMARY

PetOS is a multi-agent AI software-engineering workspace where autonomous specialist agents perform real development work while appearing as animated pixel pets inside a virtual office.

PRODUCT VISION

A user should be able to: 1. Connect a GitHub repository. 2. Create a development task. 3. Let the Captain agent analyse and divide the task. 4. Review and approve the proposed plan. 5. Watch specialist agents work visually. 6. See terminal commands, changed files, logs and test results. 7. Review the final code diff. 8. Create a pull request.

The first release must answer: Can a developer complete real software-development work using PetOS?

MVP SCOPE

Required MVP capabilities include accounts and projects, development workflow, initial agents (Captain, ThemeFox, BugDog), visual workspace, and shared intelligence.

ARCHITECTURE PRINCIPLES

Start with a modular monolith; PostgreSQL is the source of truth; visual state is a projection; risky actions require policy enforcement; every operation must be observable.

TECHNOLOGY STACK

Frontend: Next.js App Router, React, TypeScript, Tailwind CSS, shadcn/ui, Framer Motion, PixiJS, Zustand, TanStack Query, Zod. Backend: Python, FastAPI, Pydantic, SQLAlchemy, Alembic, LangGraph, Temporal, WebSockets, PostgreSQL, pgvector, Redis Streams. Testing: Pytest, Vitest, React Testing Library, Playwright, Ruff, mypy or Pyright, ESLint, Prettier. Infrastructure: Docker, Docker Compose, GitHub Actions, Caddy or Traefik, OpenTelemetry, Prometheus, Grafana, Structured JSON logs.

RECOMMENDED REPOSITORY STRUCTURE

Monorepo structure includes apps/web, services/api, services/agent-worker, services/executor, packages (ui, pixel-engine, contracts, event-schema, shared-types, configuration), infra, docs, scripts, GitHub workflows, and root configuration files.

DEVELOPMENT PLAN

Fifteen phases from product specification and engineering foundation through accounts/projects, task management, pixel office, agent planning/execution, GitHub integration, memory, durable workflows, deployment, hardening, and beta release.

DEFINITION OF DONE

MVP complete when a new user can register/login, create project, connect GitHub repository, create task, receive and approve Captain plan, watch agents work, inspect live activity/output, review files/tests, approve final result, create pull request, resume failed run, and see complete audit history with real repository/code changes.

MAJOR MISTAKES TO AVOID

Avoid premature microservices, unrestricted shell access, using Redis as permanent DB, unapproved pushes, sending complete repo to model, storing secrets in prompts, overbuilding pixel systems early, too many agents, conflating LangGraph/Temporal, single provider dependency, paid AI in CI tests, treating animation as state, early Kubernetes, early deployment focus before safe execution, and measuring code output instead of acceptance criteria.

FINAL ARCHITECTURE RECOMMENDATION

Build PetOS as a modular monolith with isolated execution, not a large distributed platform. Milestone 1: user creates task, Captain generates structured plan, user approves, pixel office visualizes workflow. Milestone 2: agent safely modifies real repository, runs tests, presents reviewable Git diff. Milestone 3: PetOS creates real GitHub pull request that passes BugDog validation.